A Latent Bug in the Adaptive Subdivision Heuristic of `mandelHybrid`

Matias Saavedra

Saturday 30th May, 2026

Analyses the bug present in `0f782fd` (tag `binary-v0-buggy`) and presents the one-line fix that lands in `d5bf30c` (tag `binary-v1-bugfix`).

Resumen

This report documents a latent correctness bug in `MandelRegion::examine()`, the function at the heart of the adaptive load-balancing strategy used by `mandelHybrid`. The bug lies in the loop that computes the minimum and maximum iteration counts across the four corners of a region: by chaining the two updates with an `else if`, the original code makes max-tracking unreachable on the first iteration and on every subsequent iteration where a new minimum is found. As a consequence, regions whose largest corner value happens to be the upper-right corner are systematically misclassified as "uniform" and computed in a single pass, even when they straddle a high-iteration boundary filament. The bug coexists with—and very likely contributed to—the 15.3 s worst-case CPU region observed in the original profiling run. We describe the role of `examine()` in the hybrid renderer, walk through the failure modes of the original loop on representative corner-value patterns, and present a one-line fix that restores correct independent tracking of `minIter` and `maxIter`.

1. Context: How the Adaptive Subdivision Works

A `MandelRegion` represents a rectangular sub-area of the output image, holding both its pixel-space coordinates and the complex-plane coordinates that it maps to. Each worker thread (one driving the GPU, the rest CPU-only) pops regions from a shared `WorkQueue` and calls `MandelRegion::examine()` on them. This function is the entire load-balancing strategy of the renderer: it decides, for every region, whether to compute its pixels immediately or to subdivide it into four quadrants that will be pushed back onto the queue for other workers to pick up.

1.1. The Corner Heuristic

The cheap test that drives this decision is to sample the Mandelbrot iteration count `diverge()` at the four corners of the region (`UPPER_LEFT`, `UPPER_RIGHT`, `LOWER_LEFT`, `LOWER_RIGHT`). If the four iteration counts are close to one another, the region is assumed to be uniform—either entirely inside the Mandelbrot set or entirely outside it—and is computed as a single block. If the counts spread widely, the boundary of the set is presumed to cut through the region, so the region is split.

The corner values are stored in `cornersIter[4]`, initialized to the sentinel `UNKNOWN = -1`. When a region is subdivided, each child inherits exactly one corner value from its parent, so that no corner of the original region is recomputed by its descendants. The decision threshold is configured by the static field `MandelRegion::diffThresh` (default `0.5`) and the absolute lower bound `MandelRegion::pixelSizeThresh` (default `32768` pixels), set by `initColorMapAndThrer()`:

Código 1: mandelregion.cpp — static thresholds.

```
QRgb *MandelRegion::colormap;
double MandelRegion::diffThresh;
int MandelRegion::pixelSizeThresh;

void MandelRegion::initColorMapAndThrer (int maxV,
                                         double diffT = 0.3,
                                         int    pixT  = 2048)
{
  colormap = new QRgb[maxV];
  diffThresh = diffT;
  pixelSizeThresh = pixT;
  // ... colour map initialization ...
}
```

1.2. The Decision Rule

Once the four corner iteration counts have been computed, the decision rule is a single boolean expression:

Código 2: mandelregion.cpp — decision rule.

```
// either compute the pixels or break the region in 4 pieces
if (maxIter - minIter < diffThresh * maxIter
    || pixelsX * pixelsY < pixelSizeThresh)
  {
    compute (onGPU);
    ownerFrame->regionComplete ();
  }
else
  {
    // ... four-way split, push four children onto the queue ...
    ownerFrame->regionSplit ();
  }
```

The first disjunct, `maxIter - minIter < diffThresh * maxIter`, is the uniformity check. The second is the absolute size floor that prevents infinite subdivision near the fractal boundary. Both `minIter` and `maxIter` must therefore be computed correctly across the four corners for the heuristic to work as intended.

2. The Bug

2.1. The Original Loop

The reduction that computes `minIter` and `maxIter` from the four corner values appears immediately above the decision rule:

Código 3: mandelregion.cpp — original min/max reduction (buggy).

```
int minIter = INT_MAX, maxIter = 0;

// evaluate the corners first
for (int i = 0; i < 4; i++)
  {
    if (cornersIter[i] == UNKNOWN)
      {
        switch (i)
          {
          case (UPPER_RIGHT):
            cornersIter[i] = diverge (lowerX, upperY);
            break;
          case (UPPER_LEFT):
            cornersIter[i] = diverge (upperX, upperY);
            break;
          case (LOWER_RIGHT):
            cornersIter[i] = diverge (lowerX, lowerY);
            break;
          default:          // LOWER_LEFT
            cornersIter[i] = diverge (upperX, lowerY);
          }
      }
    if (minIter > cornersIter[i])
      minIter = cornersIter[i];
    else if (maxIter < cornersIter[i])
      maxIter = cornersIter[i];
  }
```

2.2. Why the `else if` Is Wrong

Tracking the running minimum and the running maximum of a sequence are *independent* operations: a single value can simultaneously be the smallest seen so far and (trivially, if it is the only sample) the largest seen so far. The original code chains the two updates with an `else if`, which says "only consider updating `maxIter` if we did not just update `minIter`". Whenever a sample lowers the minimum, the max-update is silently skipped on that sample.

The most damaging case is the first iteration. Because `minIter` is initialized to `INT_MAX`, the very first corner always triggers the min branch, and the max branch is never entered. The first corner's value therefore never has the chance to set `maxIter`.

2.3. Failure Walk-Throughs

We analyse three representative patterns of corner values. In each case, recall that `minIter` starts at `INT_MAX` and `maxIter` starts at `0`, and corners are visited in the

order UPPER_RIGHT, UPPER_LEFT, LOWER_RIGHT, LOWER_LEFT as encoded by the constants in mandelregion.h.

Case A — the maximum is at the first corner.

Corner values: [8000, 50, 60, 70].

- $i = 0$: INT_MAX > 8000 is true $\Rightarrow$ minIter $= 8000$. The else if is skipped. maxIter stays 0.
- $i = 1$: $8000 > 50 \Rightarrow$ minIter $= 50$. Skipped.
- $i = 2$: $50 > 60$ false $\Rightarrow 0 < 60$ true $\Rightarrow$ maxIter $= 60$.
- $i = 3$: $50 > 70$ false $\Rightarrow 60 < 70$ true $\Rightarrow$ maxIter $= 70$.

Final: minIter $= 50$, maxIter $= 70$. The real maximum of 8000 has been discarded. The decision rule evaluates as $70 - 50 = 20 < 0.5 \cdot 70 = 35$ — *the region is classified as uniform and computed in a single block*, even though the corner values actually span by a factor of 160. If the missing high-iteration corner is representative of a high-iteration filament passing through the upper-right area, this is exactly the misclassification pattern that produces a worst-case CPU region.

Case B — all four corners equal (genuinely uniform interior).

Corner values: [100, 100, 100, 100].

- $i = 0$: INT_MAX $> 100 \Rightarrow$ minIter $= 100$. Skipped. maxIter stays 0.
- $i = 1$: $100 > 100$ false $\Rightarrow 0 < 100$ true $\Rightarrow$ maxIter $= 100$. *Rescued.*
- $i = 2, 3$: no change.

Final: minIter $=$ maxIter $= 100$. Spread is zero, the region is correctly classified as uniform. This case—where the first corner's value happens to repeat at the second corner—is the only reason the bug is not catastrophic in practice. For Mandelbrot images, the vast majority of uniform regions are interior tiles where all four corners hit MAXITER, and they self-rescue at $i = 1$.

Case C — strictly decreasing corners.

Corner values: [400, 300, 200, 100].

- $i = 0$: minIter $= 400$, maxIter stays 0.
- $i = 1$: $400 > 300 \Rightarrow$ minIter $= 300$. Skipped.
- $i = 2$: $300 > 200 \Rightarrow$ minIter $= 200$. Skipped.
- $i = 3$: $200 > 100 \Rightarrow$ minIter $= 100$. Skipped.

Final: minIter $= 100$, maxIter $= 0$. The decision rule evaluates as $0 - 100 = -100 < 0.5 \cdot 0 = 0$ — true. The region is classified as uniform and computed in a single block. Note also that the right-hand side diffThresh $\cdot$ maxIter collapses to 0 whenever the bug discards the true maximum, making the decision rule degenerate: *any* negative left-hand side passes, and the spread can never look "large" relative to a maximum of zero.

2.4. Probability of Triggering the Bug

Given four random corner samples, the maximum lies at position 0 (visited first) with probability 1/4 assuming uniformly random permutation of ranks. So roughly one region in four has its maximum silently dropped purely from corner-visit order, before considering the spatial correlations of the Mandelbrot iteration field. Most of those regions still happen to be classified correctly, because Mandelbrot iteration counts are highly correlated across nearby points and Case B patterns dominate. But the regions that *do* contain a single high-iteration corner at position 0 are exactly the regions that should have been split—and they are precisely the regions that produce the long-tail outliers in the profile.

3. The Fix

The repair is to use two independent `if` statements:

Código 4: mandelregion.cpp — corrected min/max reduction.

```
// Track min and max independently. The original code chained these
// with `else if`, which made max-tracking unreachable whenever the
// current sample also lowered the min -- most notably on i==0, where
// minIter starts at INT_MAX so the min branch always fires and
// maxIter is never updated from the first corner.
if (cornersIter[i] < minIter)
  minIter = cornersIter[i];
if (cornersIter[i] > maxIter)
  maxIter = cornersIter[i];
```

After the fix:

- Case A correctly yields `minIter` = 50, `maxIter` = 8000. The decision rule evaluates as $7950 < 0.5 \cdot 8000 = 4000$ — false — and the region is split, as it should be.
- Case B still correctly yields 100 = 100, and the region is computed in one block.
- Case C now yields `minIter` = 100, `maxIter` = 400. The decision rule evaluates as $300 < 0.5 \cdot 400 = 200$ — false — and the region is split.

The fix adds one comparison per corner (four extra cheap comparisons per call to `examine()`). Compared to the cost of even a single `diverge()` corner evaluation, which runs up to `MAXITER` multiplications and additions, this is negligible.

4. Connection to the 15.3-Second Outlier

The original profiling report identified the worst-case CPU region of 15.3 s as a misclassified boundary region, and attributed it to the intrinsic weakness of the four-corner heuristic: a fractal filament crossing the region's interior without touching any of the four corners. That explanation is plausible and likely contributes to outliers in general.

However, the min/max tracking bug provides a second, simpler explanation that should be ruled out first: a region whose only high-iteration corner happened to be `UPPER_RIGHT`—visited at $i = 0$—will have that corner's value silently dropped from `maxIter`, with the decision rule then degenerating to "always uniform" (Case C above) or to a spread that looks artificially small (Case A). Either outcome classifies the region as uniform and dispatches it directly to `compute()`.

In the original profile (100 frames, 1920×1080, 12 threads), the bug would have been triggered for an estimated $\sim 25\%$ of regions on each `examine()` call (the probability that the sample at $i = 0$ is also the maximum), although only a small fraction of those would result in observable misclassification. Re-running the profile with the corrected loop is the cleanest way to disentangle the bug's contribution from the heuristic's intrinsic weakness. Any remaining outliers after the fix can then be attributed to the heuristic itself, justifying the further improvements suggested in the original report (lower `diffThreshold`, edge-midpoint sampling, cardioid/bulb interior certificates).

5. Summary

- `MandelRegion::examine()` is the load-balancing policy of `mandelHybrid`: it samples four corners, computes the spread of their iteration counts, and either subdivides the region or computes its pixels.
- The original loop chains the min-update and the max-update with an `else if`, making max-tracking unreachable whenever the current sample also lowers the min.
- On the first iteration this is unconditional: the first corner's value is never considered as a potential maximum.
- In Case A (max at corner 0 only) the bug discards the true maximum and the region is wrongly classified as uniform. In Case C (strictly decreasing corners) `maxIter` remains 0, the decision rule degenerates, and the region is again wrongly classified as uniform.
- Case B (all corners equal) is the only reason the bug is tolerable in practice: most uniform interior tiles have all four corners at `MAXITER`, so the second iteration rescues `maxIter`.
- The fix is one line: replace `else if` with a second independent `if`, so that every sample updates both `minIter` and `maxIter`.
- The bug is a strong candidate for at least part of the 15.3-second outlier identified in the previous profiling report. Re-profiling with the fix in place will clarify how much of the load-balance imbalance was attributable to this bug versus to the intrinsic weakness of the four-corner heuristic.